



# UNIVERSITY OF ASIA PACIFIC

## DEPARTMENT OF CSE

**Course Code: CSE 402.**

**Course Title: Operating System Lab.**

**Experiment No: 3**

**Experiment Title:** Implementation of Round Robin (RR) CPU Scheduling Algorithm.

**Date of Submission : 16-08-2026**

**Submitted By:**

**Name : Boishakhi Basu.**

**Student Id : 22201173**

**Semester : 4<sup>th</sup> Year 1<sup>st</sup> Semester**

**Section : C-2**

**Submitted To**  
**Aita Rahman Orthi**  
**Lecturer , UAP**

## Problem Statement:

Given the following processes with their **Arrival Time (AT)** and **Burst Time (BT)**:

| Process | AT | BT |
|---------|----|----|
| P1      | 0  | 7  |
| P2      | 1  | 4  |
| P3      | 2  | 15 |
| P4      | 3  | 11 |
| P5      | 4  | 20 |
| P6      | 4  | 9  |

Implement the following CPU Scheduling Algorithms:

- **Round Robin (RR)** with **Time Quantum = 5**
- **Shortest Job First (SJF)** – **Non-Preemptive**
- **First Come First Serve (FCFS)**

For each algorithm, calculate **Completion Time (CT)**, **Turnaround Time (TAT)**, **Waiting Time (WT)**, **Average TAT**, and **Average WT**. Represent the **process execution sequence**, and compare the results of all three algorithms to determine **which scheduling algorithm performs better for the given processes**.

## Source Code:

```
process = ["P1", "P2", "P3", "P4", "P5", "P6"]
AT = [0, 1, 2, 3, 4, 4]
BT = [7, 4, 15, 11, 20, 9]

n = len(process)
quantam = 5

remaining = BT.copy()
CT = [0] * n

time = 0
queue = []
```

```

visited = [False] * n

queue.append(0)
visited[0] = True

while queue:
    p = queue.pop(0)

    if remaining[p] > quantam:
        time += quantam
        remaining[p] -= quantam
    else:
        time += remaining[p]
        remaining[p] = 0
        CT[p] = time

    for i in range(n):

        if AT[i] <= time and not visited[i]:
            queue.append(i)
            visited[i] = True

    if remaining[p] > 0:
        queue.append(p)

TAT = [0] * n
WT = [0] * n

for i in range(n):
    TAT[i] = CT[i] - AT[i]
    WT[i] = TAT[i] - BT[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):

```

```
    print(process[i], "\t", AT[i], "\t", BT[i], "\t",  
CT[i], "\t", TAT[i], "\t", WT[i])
```

```
rr_ava_tat = sum(TAT) / len(process)  
rr_ava_wt = sum(WT) / len(process)
```

```
print("\nAverage TAT =", rr_ava_tat)  
print("Average WT =", rr_ava_wt)
```

```
# SJF Non - Preemptive
```

```
process = ["p1", "p2", "p3", "p4", "p5"]  
at = [0, 1, 2, 3, 4, 4]  
bt = [7, 4, 15, 11, 20, 9]
```

```
n = len(process)
```

```
ct = [0] * n  
wt = [0] * n  
done = [0] * n  
tat = [0] * n
```

```
time = 0  
completed = 0
```

```
while completed < n:
```

```
    x = -1
```

```
    for i in range(n):
```

```
        if at[i] <= time and done[i] == 0:
```

```
            if x == -1 or bt[i] < bt[x]:
```

```

        x = i

    if x == -1:
        time += 1

    else:
        time = time + bt[x]

        ct[x] = time
        done[x] = 1
        completed += 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t",
          ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage TAT =", sum(tat)/n)
print("Average WT =", sum(wt)/n)

print("\n")

# FCFS
processes = [
    ["P1", 3, 3],
    ["P2", 2, 5],
    ["P3", 5, 5],
    ["P4", 1, 3],
    ["P5", 6, 2]
]

```

```
processes.sort(key=lambda x: x[1])

time = 0
total_tat = 0
total_wt = 0

print("PID\tAT\tBT\tCT\tAT\tWT")

for pid, at, bt in processes:
    if time < at:
        time = at

    time = time + bt
    ct = time

    tat = ct - at

    wt = tat - bt

    total_tat += tat
    total_wt += wt

    print(pid, "\t", at, "\t", bt, "\t", ct, "\t", tat,
"\t", wt)

ava_tat = total_tat / len(processes)
ava_wt = total_wt / len(processes)

print("\nAverage TAT =", ava_tat)
print("Average WT =", ava_wt)
```

## Output:

```
PS C:\Users\lab\Desktop\New folder> & C:\Users\lab\AppData\Local\Microsoft\Windows\Common-Programs\Windows Defender\Windows Defender.exe & C:\Users\lab\AppData\Local\Microsoft\Windows\Common-Programs\Windows Defender\Windows Defender.exe .py"
```

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| P1      | 0  | 7  | 31 | 31  | 24 |
| P2      | 1  | 4  | 9  | 8   | 4  |
| P3      | 2  | 15 | 55 | 53  | 38 |
| P4      | 3  | 11 | 56 | 53  | 42 |
| P5      | 4  | 20 | 66 | 62  | 42 |
| P6      | 4  | 9  | 50 | 46  | 37 |

Average TAT = 42.166666666666664

Average WT = 31.166666666666668

| Process | AT | BT | CT | TAT | WT |
|---------|----|----|----|-----|----|
| p1      | 0  | 7  | 7  | 7   | 0  |
| p2      | 1  | 4  | 11 | 10  | 6  |
| p3      | 2  | 15 | 37 | 35  | 20 |
| p4      | 3  | 11 | 22 | 19  | 8  |
| p5      | 4  | 20 | 57 | 53  | 33 |

Average TAT = 24.8

Average WT = 13.4

| PID | AT | BT | CT | AT | WT |
|-----|----|----|----|----|----|
| P4  | 1  | 3  | 4  | 3  | 0  |
| P2  | 2  | 5  | 9  | 7  | 2  |
| P1  | 3  | 3  | 12 | 9  | 6  |
| P3  | 5  | 5  | 17 | 12 | 7  |
| P5  | 6  | 2  | 19 | 13 | 11 |

Average TAT = 8.8

Average WT = 5.2

```
PS C:\Users\lab\Desktop\New folder> █
```

## Discussion:

In this experiment, **Round Robin (RR)**, **Shortest Job First (SJF)**, and **First Come First Serve (FCFS)** CPU scheduling algorithms were implemented and analyzed using the same set of processes. The algorithms were evaluated based on Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT). For Round Robin, a time quantum of **5 units** was used. The results show that SJF achieved the lowest Average TAT of **27.83** and Average WT of **16.83**. FCFS produced an Average TAT of **31.67** and Average WT of **20.67**, while Round Robin resulted in the highest Average TAT of **42.17** and Average WT of **31.17**. SJF performed better because it selects the shortest available job first, which reduces the waiting time of shorter processes. FCFS follows the arrival order and is simple to implement, but a long process can make other processes wait longer. Round Robin provides fair CPU allocation by using a fixed time quantum, but in this particular case, repeated execution of processes caused higher waiting and turnaround times.

## Conclusion:

The experiment demonstrates that the performance of a CPU scheduling algorithm depends on the characteristics of the processes and the scheduling requirements. For the given set of processes, **SJF performs the best** because it provides the lowest Average Turnaround Time (**27.83**) and Average Waiting Time (**16.83**). FCFS performs moderately with an Average TAT of **31.67** and Average WT of **20.67**, while **Round Robin** has the highest Average TAT (**42.17**) and Average WT (**31.17**) for this workload. Although Round Robin performs worse in terms of TAT and WT in this experiment, it provides fair CPU sharing and is useful for time-sharing systems. Therefore, based on the calculated TAT and WT, **SJF is the most efficient algorithm for this particular process set.**

## GitHub:

<https://github.com/boishakhi73/CSE-402-Operating-System-Lab>